Formal Verification for Firmware

A formal-verification finding in NVIDIA OpenSMA, confirmed and fixed upstream

Lucas C. Cordeiro

University of Manchester
lucas.cordeiro@manchester.ac.uk

May 2026

Reference: `verification/REPORT.md` | Issue: NVIDIA/OpenSMA#1

Executive summary

In one paragraph

We applied **formal verification** to **22 modules** of NVIDIA OpenSMA firmware using the open-source ESBMC model checker. The tool **produced a counterexample** showing one MCTP control path can be driven into an out-of-bounds array access within the harness's modelled bound (**F-1**, filed as NVIDIA/OpenSMA#1; **confirmed and fixed upstream by NVIDIA**). Seven further findings were confirmed (three reachable, five latent). All in **sub-second proofs**, with **no test cases written by hand**.

What it is

A solver-based proof that a program **cannot** reach a bad state — over **all inputs**, not a hand-picked few.

What we found

F-1: Control SetEpId Request can **abort the firmware** via `set_cur_eid()`. A path testing missed for years.

Why it matters

Firmware bugs are **expensive to recall**. Formal proofs catch whole **classes** of bugs, pre-silicon and pre-ship.

The F-1 bug — a four-line view

```
// pdk-mctp-platforms-router-plat.cpp:34
void set_cur_eid(RoutingTable& rt,
                 Packet::InterfaceType iface,
                 uint8_t eid)
{
    rt.ec.cur_eid.at(iface) = eid;  // (*)
}
```

Source: OpenSMA, © NVIDIA Corporation, Apache-2.0.

The setup

- `cur_eid` is sized `UsEnd = 2`.
- Validator rejects only `iface` $\geq$ `End = 18`.
- Any `iface` $\in [2, 17]$ **passes validation**, then aborts at $(*)$.
- Sibling `get_cur_eid()` has the guard; `set_cur_eid()` **does not**.

Behaviour under production flags

With `-fno-exceptions -fno-rtti`, `std::array::at(OOB)` calls `abort()` — the firmware terminates (SIGABRT, exit 134).

Trigger

An MCTP Control SetEpId Request whose `packet_interface` falls in the gap drives the dispatch path into the OOB access. Reported upstream as **NVIDIA/OpenSMA#1**; **confirmed and fixed by NVIDIA** (2026-05-11) — the bound check is now mirrored into `set_cur_eid()` as recommended.

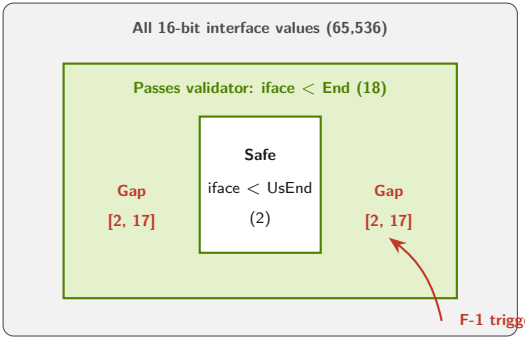
Why traditional testing missed F-1

The trigger lives in a tiny slice of a huge input space

- `InterfaceType` is a `uint16_t` — 2^{16} values.
- Only 16 trigger the abort: $[2, 17]$.
- The packet must **also** pass ~ 9 unrelated header guards (version, msg type, command, request bit, ...).
- Random / fuzz testing must hit all of those at once.

The asymmetry is structural

- The sibling getter *does* have the guard — easy to assume the setter does too.
- Unit tests cover the single-interface happy path well.



ESBMC explores **the entire green zone** symbolically; fuzzing samples a handful of points.

What is formal verification, in engineering terms?

The idea

Translate the source code and a property ("no out-of-bounds access", "no overflow", "output is in $[0, 100]$") into **logical equations**. Hand them to a solver. The solver either **proves the property holds for every input**, or returns a **concrete counterexample** that breaks it.

Three outcomes:

- **SAFE** — proven correct under the modelled bound.
- **FAILED** — counterexample produced (concrete input + trace + failing assertion).
- **UNKNOWN** — solver gave up (rare for firmware-sized code).

Properties checked by default:

- Buffer / array out-of-bounds
- Null and dangling pointer dereferences
- Integer over- / underflow, div-by-zero
- Memory leaks, use-after-free
- Undefined-behaviour shifts, signed wraps
- Loop exhaustion — every loop is **proven** to terminate within its stated bound (not silently truncated)
- *User-defined* functional contracts

A counterexample is not a guess — it's a **concrete input** the engineer can paste into a debugger and reproduce.

How ESBMC proved F-1 — in three states

Setup

- Harness constructs a Control SetEpId Request; 9 header fields constrained to pass `Validator::validate()`.
- `iface_val` left *nondeterministic* in $[2, 18)$.
- Calls production `on_set_endpoint_id()` unmodified.

Solver result

- 275 verification conditions generated.
- Bitwuzla returns SAT in < 0.01 s.

Production confirmation

- Generated test case compiled and run natively.
- Process exits with **SIGABRT (signal 6)**.

```
ESBMC counterexample (abridged):

  State 5   iface_val = 2
            (passes assume >= UsEnd && < End)

  State 9   valid = 1
            (Validator::validate() returns true)

  State 10  Violated property:
            std::array::at out of range
            [i::0 < 2 fails]
            at set_cur_eid()
            -> cur_eid.at(2) on size-2 array

VERIFICATION FAILED
```

End-to-end proof from validator entry through dispatch to OOB write. **No code modification, no manual test input.**

Formal verification vs. traditional testing

	Traditional testing	**Formal verification**
Coverage of input space	A finite, hand-picked sample (or random for fuzz)	**Symbolic — every input in the modelled range**
Bug discovery	Finds bugs you anticipated	**Finds bugs no one thought to test**
Bug evidence	Failing test + stack trace	**Auto-generated input + full executable trace**
Concurrency	Schedule-dependent — bug only on rare interleavings	**Explores all thread interleavings symbolically**
Cost per defect (late)	High — found in QA / field	**Low — found pre-commit**
Maintenance	Tests rot; coverage drifts	Properties live next to the code; CI re-proves on every PR
Confidence claim	"We tested it"	**"It cannot reach this state for any input"**
Where it fails	Sparse coverage of the rare path	**Large heap-aliasing problems;** known-finite loops are proven exhausted via unwinding assertions

*The two are complementary, not competing. FV is most valuable on **security-critical, hard-to-test, deeply-conditional** code paths — exactly the firmware control plane.*

LLMs vs. formal verification — complement, not replacement

Where LLMs win (and where they break)

- Strong on **small (20–100 LOC) snippets**; broad CWE coverage incl. non-formal classes (SQL injection, weak crypto, hard-coded credentials).
- Honest data: GPT-o3-mini scores **955** on CASTLE vs. ESBMC **697** on the same 250 micro-benchmarks (TASE 2025).
- **But** on a 400+ LOC merged file LLMs **hallucinate** false positives and **miss hidden vulnerabilities**.
- **Non-deterministic**: same prompt → different answers. No counterexample. No proof. No CI gate.

Where ESBMC wins (the firmware case)

- Every alarm ships with an **executable counterexample** — auditable, replayable, deterministic.
- FP rate is **structurally bounded**: a counterexample either reproduces or it does not.
- Scales to **whole-TU memory-safety properties**; F-1 was found this way in sub-second proofs.
- Re-runs **identically on every PR** — a real CI gate.

Integration message

LLM in the IDE for breadth and real-time hints. **ESBMC at the CI gate** for memory-safety guarantees on firmware control paths. CASTLE's best two-tool combo is FV + classical SAST (**ESBMC + CodeQL = 831**), with LLMs feeding the developer loop upstream — not gating the merge.

The OpenSMA pilot at a glance

Scope (1 engineer, 9 days: 25 Apr – 3 May 2026)

- **22 firmware modules**, **~7,900 LOC** — roughly **7%** of the first-party C++ in OpenSMA.
- Targeted the MCTP/NSM control plane, I2C sensor drivers, and security-relevant validators; skipped vendored `libspdm` (~280 k LOC) and FreeRTOS / MCU SDK glue (~45 k LOC).
- Three phases: language-level safety (with loop-exhaustion assertions), *k*-induction over functional contracts, and a separate volatile-check phase — plus negative tests that produce the F-* counterexamples. All proofs **sub-second**.

Findings (8 active, 3 retracted)

- **Confirmed reachable (3): F-1** abort path in MCTP dispatch (**NVIDIA/OpenSMA#1 — confirmed and fixed upstream by NVIDIA, 2026-05-11**); F-6 unchecked `mode` byte stored in error-injection config; F-7 dirty `portRecoveryResp` written before validation.
- Latent today (5): F-4 UB shift in `operator""_bit`; F-5 missing bounds guard in `set_bit` on the 8-byte event bitmask; F-8 fixed-index `gpio_ei_entries[16]` OOB; F-10 silent 125 °C substitution in busbar threshold setter (dead code: `BusBarTempSensorNum = 0` on all current platforms); F-13 negative-percent wrap in debug-telemetry SMA (system proof shows upstream `std::clamp` blocks it).
- **Retracted on review:** F-2, F-3, F-14.

Modules covered

MCTP packet parser | MCTP routing helpers | MCTP dispatch | MCTP validator state machine | Fixed-point arithmetic | Saturating arithmetic | NSM type-2 PCIe-link reset | NSM type-3 sensor availability | NSM type-5 field validators | NSM event bitmask | Telemetry sensor-ID lookup | SPI byte-buffer (de)serialisation | I2C CRC-8 | User-defined integer literals | NTC thermistor table | Power-smoothing presets | FRU utilities | SoC SMA filter | Debug-telemetry SMA | PCA9555 GPIO expander | EMC1812 temp sensor | TMP1075 temp sensor | TMP461 temp sensor

Verification budget

~5,000 verification conditions across Phase 1, Phase 2 (*k*-induction), volatile-check, and negative tests — all solved sub-second on Bitwuzla 0.8.2. Reproducible with `make all`.

Tangible value: ROI for firmware

Direct savings

- One F-1-class issue caught **pre-ship** avoids field debug, OTA / RMA cost, disclosure / CVE handling, brand impact.
- Industry benchmark: a defect found post-release costs **30–100×** a defect found in requirements / early design.

Speed of iteration

- Sub-second proofs ⇒ PR CI with **linter-class latency**.
- Unlike a linter, the verdict is a **mathematical proof over all inputs**, not a pattern match.
- CASTLE (TASE 2025): ESBMC tops 13 SAST tools on 250 CWE programs — **12 false positives vs. 259–1,104**.

Compounding value

- Properties written once; **re-proven on every commit, forever** — linters only know rules their authors shipped.
- Catches regressions as the code evolves — testing's biggest blind spot.
- Property files double as **executable specifications** for new engineers.

Safety & certification leverage

- Counts toward **ISO 26262 / IEC 61508** unit-level evidence.
- Required or recommended in DO-178C (avionics), EN 50128 (rail), and emerging automotive ASIL-D expectations.

Cost ratio: Boehm, *Software Engineering Economics* (Prentice-Hall, 1981), Fig. 4-2, p. 40; RTI, *The Economic Impacts of Inadequate Infrastructure for Software Testing*, NIST Planning Report 02-3 (2002), Tables 1-1 and 5-2. F-1 contains no anti-pattern targeted by clang-tidy / cppcheck / MISRA-C 2012.

Risk reduction & strategic differentiation

Bug classes formal verification eliminates

- Memory-safety violations (OOB, UAF, null deref).
- Integer over- / underflow on critical paths.
- Concurrency races — bugs hidden in one interleaving out of millions; tests almost never reproduce them.
- Protocol-state-machine violations.
- Implicit standard-conformance issues (shift UB, etc.).

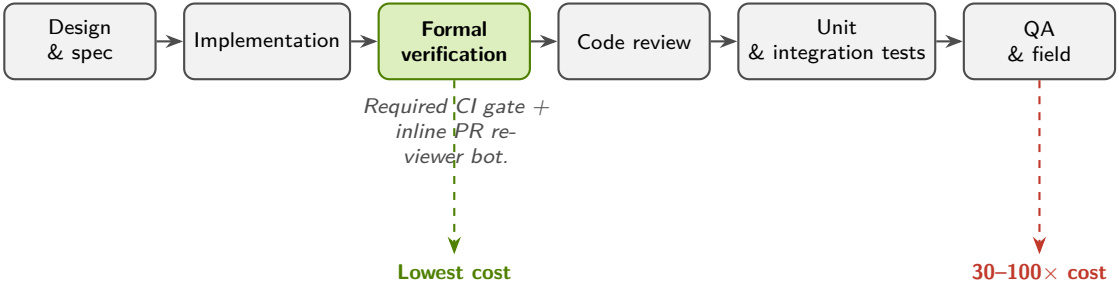
These are the bug classes that drive

- firmware-borne CVEs,
- silent data corruption,
- watchdog-induced reboots in the field.

Differentiation for NVIDIA

- Datacenter customers increasingly ask for **provable** security properties on management firmware.
- Hyperscaler procurement teams already require formal evidence for boot / attestation paths.
- "**Verified** by formal methods" is a credible market message — not marketing language.
- Public examples of FV on critical firmware include AWS (s2n), Microsoft (Azure RTOS), Google (Android Verified Boot, Titan-M), Apple (sepOS), and Arm (CCA).

Where formal verification fits in the firmware workflow



Formal verification slots in **between implementation and review**, catching defects when fixing them is cheapest. ESBMC runs as a **required CI check** *and* posts each counter-example as an inline PR review comment — a CodeRabbit-style bot, but with **proofs in place of heuristics**: every comment carries a **concrete reproducer**, not a hunch. Counterexamples become regression tests automatically — feeding the existing test pyramid.

Cost ratio: Boehm, *Software Engineering Economics* (Prentice-Hall, 1981), Fig. 4-2, p. 40; RTI, *The Economic Impacts of Inadequate Infrastructure for Software Testing*, NIST Planning Report 02-3 (2002), Tables 1-1 and 5-2.

Tooling investment & ecosystem position

What was used

- **ESBMC 8.2** — open source, actively maintained by the SSV Lab (Univ. of Manchester / Federal Univ. of Amazonas), with contributors from Southampton and Stellenbosch.
- **Bitwuzla** SMT solver — state-of-the-art on bitvector / array theories.
- Tool footprint: a single binary + the existing build system.

What was contributed back

- **20+ ESBMC bug reports / fixes** filed during the pilot — most merged upstream.
- Hardens the tool for everyone using it on C++20 firmware (including future NVIDIA work).

What this looks like at scale

- Property files live next to source headers.
- $\sim$1 hour per new module to set up.
- CI gate: PR fails if a property regresses.
- No license cost. No vendor lock-in.

Pricing model: pay for services, not the tool

- **Foundation:** priority bug-fix queue + quarterly training — low-commitment on-ramp.
- **Per-module:** fixed-price end-to-end verification (harness, properties, CI, 12-mo maintenance).
- **Partnership retainer:** embedded engineers, PR-bot / certification kits, ESBMC roadmap influence.
- Tool stays **open source — no licence, no lock-in**; fits NVIDIA's academic-research sponsorship pattern.

Recommendation

Three concrete next steps

1. **Land the eight finding fixes.** F-1, F-6, F-7 reachable; F-4, F-5, F-8, F-10, F-13 latent (one-line patches; harnesses in tree).
2. **Add the OpenSMA verification suite to CI.** `make all` runs in seconds; gate every PR on it. Catches the next F-1 the day it's introduced.
3. **Pick one new NVIDIA firmware module to verify next quarter.** Choose a security-critical surface (boot, attestation, key management, MCTP / NSM dispatch elsewhere). Re-evaluate ROI with internal numbers in 90 days.

What we are asking for

- Engineering sponsorship to merge fixes & enable CI.
- One firmware module per quarter as a verification target.
- A 90-day check-in to review hit rate vs. effort.

What it costs

- Open-source tooling: \$0.
- CI compute: negligible (sub-second proofs).
- Engineering effort: $\sim$0.5 FTE to scale to a second module + maintain CI.

Summary in three lines

1. Formal verification **produced a counterexample for a real abort path** in OpenSMA that manual review and unit testing had missed — **confirmed and fixed upstream by NVIDIA** (NVIDIA/OpenSMA#1).
2. It did so in **seconds**, on a laptop, with **open-source tooling** and **no production code changes**.
3. Adopting it as a CI gate is a **low-cost, high-leverage** way to harden NVIDIA firmware against a whole class of field-stopping defects.

Thank you.

Full report: `verification/REPORT.md`
Reproduce: `cd verification && make all`

Backup — The validator gap, in numbers

Symbol	Value	Where defined
`Interface::UsEnd`	2	`Packet.h` (size of `cur_eid` array)
`Interface::End`	18	`Packet.h` (validator's upper bound)
Gap range	$[2, 17]$	16 values that pass validation but OOB the array
`InterfaceType` type	`uint16_t`	full domain 2^{16} values

Two equally good fixes:

- Add `if (interface >= UsEnd) return;` to `set_cur_eid()` — match the guard that `get_cur_eid()` already carries.
- Tighten `Validator::validate()` to reject `interface >= UsEnd` instead of `>= End` — closes the gap at the validator boundary.

Backup — Why ESBMC, specifically

- **Open source.** No license barrier to internal adoption or experimentation.
- **C++20-aware**, with active maintenance — most defects we encountered during the pilot were fixed upstream within days.
- Uses industrial-strength SMT solvers (**Bitwuzla**, **Z3**). No bespoke proof language.
- **Counterexample-first** workflow: failing properties produce concrete inputs that engineers can reproduce immediately.
- Generates **CTest-compatible test cases** — counterexamples become regression tests automatically.
- **k-induction** support for proving loop / state-machine invariants without unbounded unrolling.
- Backed by 15+ years of academic research and supported by **ARM, AWS, Intel, Samsung, Motorola Mobility, Nokia Institute of Technology**, the Ethereum and Rust Foundations, EPSRC, UKRI, EU Horizon 2020, and Brazilian agencies CNPq / CAPES / FAPEAM.

Backup — ESBMC vs. static analysis: independent benchmark

CASTLE (TASE 2025): 250 C programs, 25 CWEs; 13 static analysers, 2 formal verifiers, 10 LLMs evaluated head-to-head.

Tool	TP	FP	P	R	CASTLE
ESBMC (FV)	53	**12**	82%	35%	**697**
CodeQL	35	43	45%	23%	600
Cppcheck	18	5	78%	12%	405
Clang Analyzer	13	2	87%	9%	381
Splint	23	1,029	2%	15%	−600
CodeThreat	21	1,104	2%	14%	−710
GPT-o3-mini (LLM)	121	72	63%	81%	**955**
DeepSeek R1 (LLM)	133	163	45%	89%	888
GPT-4o (LLM)	113	141	44%	75%	814
LLAMA 3.1 8B (LLM)	56	374	13%	37%	245

- **Top reasoning LLMs beat all SAST + FV tools** on 20–100 LOC snippets — but accuracy **collapses past ∼400 LOC** and is **non-deterministic**; ESBMC's 12 FPs come with reproducible counterexamples.
- ESBMC is the **highest-scoring non-LLM tool**; best two-tool combo is **ESBMC + CodeQL = 831** (+134 over ESBMC alone) — complementary, not competing.
- SAST tools split into high-precision / low-recall (Cppcheck, Clang) or runaway-FP (Splint, CodeThreat).

Dubniczky et al., *CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs towards CWE Detection*, TASE 2025, Table 3.

Dataset: `github.com/CASTLE-Benchmark`.